# 📘 Manual de commands.cpp

## ➡️ Comprobación y lectura de comandos UART

El sistema recibe comandos desde el puerto Serial1. Primero se verifica si hay datos disponibles y luego se leen hasta el fin de línea (\n), eliminando espacios y caracteres extra (\r, \n).

🧩 **Implementación:**

```
bool commandAvailable() // indica si hay datos por leer.
{
    return Serial1.available();
}

String receiveCommand() // devuelve el comando limpio como String listo
para procesar.
{
    String cmd = Serial1.readStringUntil('\n'); // lee hasta fin de linea
\n
    cmd.trim(); // elimina espacios y \r\n
    return cmd;
}
// Windows: \r\n → vuelve al inicio y baja de línea → Enter completo
// Linux / macOS: \n → solo baja de línea
// Mac antiguos (antes de OS X): \r → solo retorno al inicio
```

## ➡️ Mapeo de comandos a enumeración

Los comandos entrantes en String se convierten a un tipo Command (enum) definido en config.h para un procesamiento más seguro y rápido. Esto evita errores de comparación repetidos.

🧩 **Implementación:**

```
Command parseCommand(const String &cmd)
{
    if (cmd == "STATUS")        return CMD_STATUS;
    if (cmd == "RESET_ERRORS")  return CMD_RESET_ERRORS;
    if (cmd == "HOME_MOTOR1")   return CMD_HOME_MOTOR1;
    if (cmd == "HOME_MOTOR2")   return CMD_HOME_MOTOR2;
    if (cmd == "GET_ANGLE_1")   return CMD_GET_ANGLE_1;
    return CMD_UNKNOWN;
}
// Cada String tiene un enum asociado.
// Comandos desconocidos devuelven CMD_UNKNOWN → útil para enviar mensaje
de error.
```

## ➡️ Procesamiento de comandos

Se recibe un String y ejecuta la acción correspondiente según el comando mapeado.

🧩 **Implementación:**

```
void processCommand(const String &cmd)
{
    String trimmedCmd = cmd;
    trimmedCmd.trim(); // eliminar \r\n y espacios

    switch (parseCommand(trimmedCmd))
    {
        case CMD_STATUS:
            // Envía estado del homing del motor 1
            break;

        case CMD_RESET_ERRORS:
            // Resetea flags de error de los motores
            break;

        case CMD_HOME_MOTOR1:
            // Inicia homing motor 1
            break;

        case CMD_HOME_MOTOR2:
            // Inicia homing motor 2
            break;

        case CMD_GET_ANGLE_1:
            // Envía ángulo del sensor 1 en grados
            break;

        default:
            Serial1.println("UNKNOWN COMMAND");
            break;
    }
}
// Se hace un trim previo para evitar errores por caracteres extra.
// Cada comando tiene un bloque de acción claro.
// default asegura que comandos desconocidos generen respuesta al
controlador.
```

## ➡️ Ejemplos de comandos soportados

Comandos:

- STATUS Devuelve estado del homing (IDLE, HOMING IN PROGRESS, HOMING OK, HOMING ERROR)
- RESET_ERRORS Resetea flags de error de ambos motores
- HOME_MOTOR1 Inicia procedimiento de homing del motor 1
- HOME_MOTOR2 Inicia procedimiento de homing del motor 2

- GET_ANGLE_1 Lee sensor AS5600 del motor 1 y envía ángulo en grados
- Otros Devuelven UNKNOWN COMMAND

## ➡️ Filtrado de estado del homing

El comando STATUS no solo devuelve "IDLE", también considera:

- HOMING_INACTIVE → motor no está haciendo homing
- HOMING_OK → homing completado correctamente
- HOMING_ERROR → hubo error durante homing
- Cualquier otro estado → HOMING IN PROGRESS

🧩 **Implementación:**

```
if ((homingMotor1.state != HOMING_INACTIVE) &&
    (homingMotor1.state != HOMING_OK) &&
    (homingMotor1.state != HOMING_ERROR))
{
    Serial1.println("HOMING IN PROGRESS");
}
else if (homingMotor1.state == HOMING_OK)
{
    Serial1.println("HOMING MOTOR1 OK");
}
else if (homingMotor1.state == HOMING_ERROR)
{
    Serial1.println("HOMING MOTOR1 ERROR");
}
else
{
    Serial1.println("IDLE");
}
// Permite diferenciar claramente estado activo, completado y error.
// Útil para GUI o scripts que monitorean el robot.
```

## ➡️ Envío de ángulo del sensor AS5600

Similar al manual de sensors.cpp, GET_ANGLE_1 lee el ángulo en bruto (0–4095) y lo convierte a grados (0–360°), enviando solo 1 decimal.

🧩 **Implementación:**

```
uint16_t rawAngle = readAS5600Angle_1();            // Lectura directa
desde el sensor 1
float angleDegrees = (rawAngle * 360.0) / 4096.0;   // Conversión a grados
Serial1.print("ANGLE_1: ");                          // Comunicación por
UART
Serial1.println(angleDegrees, 1);
```

## 🧠 Resumen conceptual

1️⃣ Detección de comandos → commandAvailable()
2️⃣ Lectura de comandos → receiveCommand()
3️⃣ Mapeo seguro → parseCommand()
4️⃣ Procesamiento y respuesta → processCommand()
5️⃣ Filtros de estado y envío → homing y ángulo

Con esto se logra un control de motor robusto y monitoreable vía UART sin saturar el bus ni perder información crítica.